

# Clase 132 — IDA Pro y radare2

Parte: 5 — Explotación de sistemas y binarios · Fuente: *Eagle, The IDA Pro Book* · docs de radare2/rizin ⌚ Duración estimada: 120 min · Nivel: Intermedio

## Objetivo

Conocer los otros dos pilares del reversing: **IDA Pro** (estándar comercial, con Hex-Rays) y **radare2/rizin** (framework libre de línea de comandos, con Cutter como GUI). Aprenderás a moverte por un binario en ambos, a comparar sus filosofías (GUI vs REPL) y a elegir la herramienta según la tarea.

⚠ **Ética:** solo binarios propios/autorizados.

## Resultados de aprendizaje

Al finalizar, el alumno podrá:

- 1. **Navegar** un binario en IDA (grafos, pseudo-C de Hex-Rays, xrefs).
- 2. **Analizar** el mismo binario en radare2 con los comandos esenciales.
- 3. **Comparar** flujos de trabajo GUI (IDA/Cutter) vs REPL (r2).
- 4. **Renombrar/anotar** en ambas herramientas.
- 5. **Automatizar** con r2pipe.

## Temas

| # | Tema                           | Por qué importa                |
|---|--------------------------------|--------------------------------|
| 1 | IDA: vista de grafo y pseudo-C | Análisis visual potente        |
| 2 | Hex-Rays decompiler            | C legible en IDA               |
| 3 | radare2: modelo de comandos    | Todo desde el teclado          |
| 4 | aaa / afl / pdf en r2          | Análisis y desensamblado       |
| 5 | Modo visual y grafo en r2      | v , vv                         |
| 6 | Cutter (GUI de r2/rizin)       | Puente para quien viene de GUI |
| 7 | Anotar y renombrar             | Documentar el análisis         |
| 8 | r2pipe / scripting             | Automatización                 |

## Definiciones y características

- **IDA Pro:** desensamblador/decompilador comercial líder. *Clave:* Hex-Rays produce pseudo-C de alta calidad; existe IDA Free con capacidades limitadas.

- **radare2 / rizin:** framework libre orientado a comandos. *Clave:* curva pronunciada pero muy potente y scriptable; rizin es un fork con Cutter.
- **Cutter:** GUI para radare2/rizin. *Clave:* facilita la transición desde IDA/Ghidra.
- **Grafo de control de flujo (CFG):** representación visual de bloques y saltos. *Clave:* IDA y r2 ( VV ) lo muestran para entender la lógica.
- **r2pipe:** API para pilotar r2 desde Python. *Clave:* automatiza extracción y análisis en lote.

## Herramientas y preparación

```
# radare2
git clone https://github.com/radareorg/radare2 && radare2/sys/install.sh
# o: sudo apt install radare2
pip install r2pipe
# IDA Free: descargar desde hex-rays.com (opcional)
# Cutter: https://cutter.re/
```

## Laboratorio guiado

Entorno propio.

1. Abre el `crackme` en radare2 y analiza:

```
bash r2 -A ./crackme # -A ejecuta aaa (análisis) [0x...]> afl # lista funciones [0x...]> s
main; pdf # desensambla main [0x...]> VV @ main # grafo interactivo (q para salir)
```

2. Renombra y comenta en r2:

```
text afvn old_name input ; renombrar variable CCu "clave se compara aqui" @ 0x... ;
comentario
```

3. Busca cadenas y sus xrefs:

```
text izz ; todas las strings axt @ str.Enter_password ; quién referencia esa string
```

4. Si tienes IDA Free/Cutter, abre el mismo binario, ejecuta el análisis y compara la vista de grafo y el pseudo-C con lo que viste en r2.
5. Deduce la clave válida en cualquiera de las dos y verifícala ejecutando el binario.
6. Automatiza con r2pipe: script Python que imprima todas las funciones y sus tamaños:

```
python import r2pipe r = r2pipe.open("./crackme"); r.cmd("aaa") for f in r.cmdj("aflj"):
print(f["name"], f["size"])
```

## Ejercicios

1. Lista y comenta las diferencias de flujo IDA (GUI) vs r2 (REPL).
2. Usa `pdf` (pseudo-decompile) de r2 y compáralo con Hex-Rays.
3. Renombra tres variables en Cutter y en r2 puro.

4. Extrae con r2pipe todas las llamadas a `strcmp`.
5. Navega el CFG en `VV` e identifica el bloque de "éxito".
6. Exporta el análisis a un proyecto de r2 ( `Ps` ).

## Reto verificable

Analiza un `crackme` con radare2 (sin GUI) y deduce la clave válida usando solo comandos de r2.

**Criterio de aceptación:** obtienes la clave correcta trabajando desde la consola de r2 y documentas los comandos usados ( `afl` , `pdf` , `izz` , `axt` , etc.).

## Errores comunes

| Síntoma / mensaje              | Causa y cómo arreglar                                                                     |
|--------------------------------|-------------------------------------------------------------------------------------------|
| <code>afl</code> vacío         | No corriste el análisis; usa <code>-A</code> o <code>aaa</code>                           |
| Te pierdes en r2               | Empieza con <code>?</code> y <code>V</code> / <code>VV</code> ; ve incrementando comandos |
| pseudo-C de r2 pobre           | Usa Cutter/Ghidra para decompilar; r2 brilla en control fino                              |
| IDA Free no decompila esa arch | Hex-Rays por arquitectura es limitado en Free                                             |
| Cambios no persisten           | Guarda el proyecto ( <code>Ps</code> ) o la base de IDA (.idb)                            |

## Preguntas frecuentes

 **¿Cuál aprendo primero?** Ghidra (gratis, decompilador) para entender; r2 para control y scripting; IDA si tu entorno lo usa.

 **¿radare2 o rizin?** rizin es un fork más estable con Cutter integrado; los comandos son casi idénticos.

 **¿Puedo combinar herramientas?** Sí: es común triage con r2, decompilar con Ghidra/IDA y depurar con GDB.

## Referencias

- Eagle, C. *The IDA Pro Book*, 2e. No Starch Press.
- radare2 book — <https://book.rada.re/>
- Cutter — <https://cutter.re/>
- Hex-Rays / IDA — <https://hex-rays.com/>

## Material descargable

-  [Guía en PDF](#) — versión imprimible de esta clase.
-  [Presentación \(PPTX\)](#) — deck para proyectar en clase.

## Siguiendo clase

Clase 133 - Analisis estatico de binarios